ジオメトリック・インテリジェンス

ケーススタディ補助文書 03

実装スクリプト集

― 光稜精機株式会社　基準ウェハーの一周の差 ―

本書は、ケーススタディ・ノベル「光稜の巡回差」に登場した計算と図を、Python のコードで再現し、実行結果と本編の数値の対応、実装上の注意を示す文書である。

Étale Cohomology

2026 年 11 月

License: CC BY 4.0

目次

第 1 章　はじめに …… 3

1.1　本書の構成 …… 3

1.2　本作のコードの特徴 …… 3

1.3　各節の構成 …… 4

1.4　推奨ライブラリ …… 4

第 2 章　一周の差と同期問題 ― ch4_ch5_metrology.py …… 4

2.1　背景 …… 4

2.2　コード …… 4

2.3　実行結果 …… 4

2.4　解説 …… 4

2.5　実装上の注意 …… 4

2.6　参照 …… 4

第 3 章　測り方の計画、CUSUM とカルマンフィルタ ― ch4_design_ch5_cusum.py …… 6

3.1　背景 …… 6

3.2　コード …… 6

3.3　実行結果 …… 7

3.4　解説 …… 8

3.5　実装上の注意 …… 8

3.6　参照 …… 8

第 4 章　規制の両立と実物オプション ― ch6_ch7.py …… 8

4.1　背景 …… 8

4.2　コード …… 8

4.3　実行結果 …… 10

4.4　解説 …… 10

4.5　実装上の注意 …… 10

4.6　参照 …… 11

第 5 章　本編の図を描くコード ― figures/ …… 11

付録 A　推奨ライブラリ …… 12

第 1 章　はじめに

1.1　本書の構成

本書は、本編の計算を再現する実装スクリプト 3 本と、本編の図を描くコード 5 本を、章ごとに整理する。スクリプトは、GitHub のリポジトリの case-studies/corporate/semiconductor-equipment-industry-2/code/ に置いている。

スクリプト	本編の章	内容
ch4_ch5_metrology.py	第 4 章・第 5 章	一周の差の検定、いまのやり方との比較、補正値と基準ウェハーの変化の推定（本書第 2 章）
ch4_design_ch5_cusum.py	第 4 章・第 5 章	測り方の計画で区別できるか（行列の階数と D 基準）、CUSUM とカルマンフィルタ（本書第 3 章）
ch6_ch7.py	第 6 章・第 7 章	規制の両立の判定（ファルカスの補題、SAT ソルバー）、実物オプション（本書第 4 章）
figures/ の 5 本	第 2 章〜第 4 章	本編の図を描くコード（本書第 5 章）

1.2　本作のコードの特徴

前作のコードは、VAE による多様体の構築や曲率の計算など、なめらかな空間の上の計算が中心だった。本作のコードは、点と線の図の上の計算（一周の差、最小二乗法、行列の階数）と、規制の両立の判定（線形計画法、SAT ソルバー）が中心で、NumPy と SciPy と z3 だけで動く。

- **答えを仕込んだ架空のデータ。**計算に使うデータは、物語の答え（W3 は台湾から米国への輸送で+0.25 nm 変化、F2 は 40 日目の部品交換で+0.18 nm クセが変化）を仕込んで、乱数で作った架空のデータである。コードは、仕込んだ答えを、測定値だけから取り戻せるかを確かめる。
- **正解との比較。**推定値の横に正解を並べて表示し、推定の誤差を読者が確かめられるようにしている。

1.3　各節の構成

各節は「背景」「コード」「実行結果」「解説」「実装上の注意」「参照」の順に構成する。「背景」は本編のどの場面でその計算が登場したか、「解説」はコードが何を計算しているか、「実装上の注意」は実運用で修正・補強すべき点を述べる。

1.4　推奨ライブラリ

NumPy、SciPy、Matplotlib、z3-solver を前提とする。詳細は付録 A を参照。

第 2 章　一周の差と同期問題 — ch4_ch5_metrology.py

2.1　背景

出典：本編第 4 章・第 5 章。4 枚の基準ウェハーを案 B の経路で回し、各拠点で、届いたウェハーをその拠点の 2 台で測る。府中に戻った後、一周の差を検定し、W3 の変化が起きた区間と、装置ごとのクセを推定する。あわせて、業界のいまのやり方（全装置の中央値との差を補正値にする方法）と比べる。

2.2　コード

```
"""光稜精機（架空）の計測装置の合わせ込み：答えを仕込んだデータでの確認
装置：府中 F1,F2 台湾 T1,T2 米国 U1,U2 上海 S1,S2（F1 を基準=0 とする）
基準ウェハー：W1 府中→台湾→府中，W2 府中→米国→府中，W3 府中→台湾→米国→府中，W4 府中→上海→府中
仕込んだ答え：(a) W3 は台湾→米国の輸送で +0.25nm 変化　(b) F2 は 40 日目の部品交換で +0.18nm クセが変化
各拠点で、届いたウェハーをその拠点の 2 台で測る（1 回の測定は 20 点の平均。ばらつき 0.02nm）"""
import numpy as np
rng=np.random.default_rng(7)
tools=['F1','F2','T1','T2','U1','U2','S1','S2']; ti={t:i for i,t in enumerate(tools)}
off=dict(F1=0.0,F2=0.12,T1=-0.08,T2=0.05,U1=0.20,U2=-0.10,S1=0.15,S2=-0.05)
site={'F':['F1','F2'],'T':['T1','T2'],'U':['U1','U2'],'S':['S1','S2']}
routes={'W1':['F','T','F'],'W2':['F','U','F'],'W3':['F','T','U','F'],'W4':['F','S','F']}
true_cd={'W1':32.10,'W2':31.85,'W3':32.40,'W4':31.95}
change={('W3','T','U'):0.25}
sd=0.02
# 測定の記録：(ウェハー，区間番号，拠点，装置，日，値)
rec=[]; day=0
for w,r in routes.items():
    level=0.0
    for k,s in enumerate(r):
        if k>0: level+=change.get((w,r[k-1],s),0.0)
        for t in site[s]:
            o=off[t]+(0.18 if (t=='F2' and day>=40) else 0.0)
            rec.append((w,k,s,t,day,true_cd[w]+level+o+rng.normal(0,sd)))
        day+=6
# ---------- いまのやり方：ウェハーごとに、測った全装置の中央値との差を補正値とし、装置ごとに平均 ----------
med={}
for w in routes:
    vals=[v for (ww,k,s,t,d,v) in rec if ww==w]; m=np.median(vals)
    for (ww,k,s,t,d,v) in rec:
        if ww==w: med.setdefault(t,[]).append(v-m)
med_off={t:np.mean(v) for t,v in med.items()}; base=med_off['F1']
print('■ いまのやり方（中央値との差）による補正値（F1 基準，nm）')
for t in tools: print(f'  {t}: 推定 {med_off[t]-base:+.3f}   正解 {off[t]:+.3f}   誤差 {med_off[t]-base-off[t]:+.3f}')
# ---------- 一周の差：同じウェハーを府中で出発前と帰着後に、同じ装置 F1 で測った差 ----------
```

```
print('\n■ 一周の差（府中 F1 で、出発前と帰着後の差，nm）')
for w,r in routes.items():
    a=[v for (ww,k,s,t,d,v) in rec if ww==w and k==0 and t=='F1'][0]
    b=[v for (ww,k,s,t,d,v) in rec if ww==w and k==len(r)-1 and t=='F1'][0]
    z=(b-a)/(sd*np.sqrt(2)); print(f'  {w} {"→".join(r)}: {b-a:+.3f}   (z={z:+.1f}) ')
# ---------- 一周の差が有意な W3 について、どの区間で変わったかを、ほかのウェハーと組み合わせて推定 ----------
# モデル：値 = ウェハーの真値 + 区間ごとのウェハーの変化(W3 の各区間) + 装置のクセ(+F2 の段差) + ばらつき
cols=[f'cd_{w}' for w in routes]+[f'off_{t}' for t in tools[1:]]+['F2_step','W3_FT','W3_TU','W3_UF']
X=[];y=[]
for (w,k,s,t,d,v) in rec:
    row=np.zeros(len(cols)); row[cols.index(f'cd_{w}')]=1
    if t!='F1': row[cols.index(f'off_{t}')]=1
    if t=='F2' and d>=40: row[cols.index('F2_step')]=1
    if w=='W3':
        r=routes['W3']
        for kk in range(1,k+1):
            row[cols.index({('F','T'):'W3_FT',('T','U'):'W3_TU',('U','F'):'W3_UF'}[(r[kk-1],r[kk])])]=1
    X.append(row); y.append(v)
X=np.array(X); y=np.array(y)
rank=np.linalg.matrix_rank(X); print(f'\n■ 推定できるか：未知数 {len(cols)}，行列の階数 {rank}')
# W3 の区間の変化は 3 区間の和しか決まらない（ほかのウェハーが同じ区間を通らない）→ 和を 1 つの区間に割り振る候補を比べる
best=None
for keep in ['W3_FT','W3_TU','W3_UF']:
    cc=[c for c in cols if not c.startswith('W3_')]+[keep]
    Xs=X[:,[cols.index(c) for c in cc]]
    beta,res,rk,_=np.linalg.lstsq(Xs,y,rcond=None); rss=np.sum((y-Xs@beta)**2)
    print(f'  変化を {keep} に置いた場合：残差平方和 {rss:.5f}')
    if best is None or rss<best[0]: best=(rss,keep,cc,beta)
rss,keep,cc,beta=best
print(f'  → 最もよく説明するのは {keep}（変化量 {beta[cc.index(keep)]:+.3f} nm，正解 +0.250）')
print('■ 推定した装置のクセ（F1 基準，nm）')
for t in tools[1:]:
    print(f'  {t}: 推定 {beta[cc.index("off_"+t)]:+.3f}   正解 {off[t]:+.3f}')
print(f'  F2 の部品交換後の段差: 推定 {beta[cc.index("F2_step")]:+.3f}   正解 +0.180')
```

2.3　実行結果

```
■ いまのやり方（中央値との差）による補正値（F1 基準，nm）
  F1: 推定 +0.000   正解 +0.000   誤差 +0.000
  F2: 推定 +0.192   正解 +0.120   誤差 +0.072
  T1: 推定 -0.107   正解 -0.080   誤差 -0.027
  T2: 推定 +0.024   正解 +0.050   誤差 -0.026
  U1: 推定 +0.256   正解 +0.200   誤差 +0.056
  U2: 推定 -0.036   正解 -0.100   誤差 +0.064
  S1: 推定 +0.105   正解 +0.150   誤差 -0.045
  S2: 推定 -0.064   正解 -0.050   誤差 -0.014

■ 一周の差（府中 F1 で、出発前と帰着後の差，nm）
  W1 F→T→F: -0.009   (z=-0.3)
  W2 F→U→F: +0.009   (z=+0.3)
  W3 F→T→U→F: +0.210   (z=+7.4)
  W4 F→S→F: +0.040   (z=+1.4)

■ 推定できるか：未知数 15，行列の階数 15
  変化を W3_FT に置いた場合：残差平方和 0.02980
  変化を W3_TU に置いた場合：残差平方和 0.00319
  変化を W3_UF に置いた場合：残差平方和 0.04327
  → 最もよく説明するのは W3_TU（変化量 +0.230 nm，正解 +0.250）
■ 推定した装置のクセ（F1 基準，nm）
  F2: 推定 +0.124   正解 +0.120
  T1: 推定 -0.076   正解 -0.080
  T2: 推定 +0.055   正解 +0.050
  U1: 推定 +0.192   正解 +0.200
  U2: 推定 -0.101   正解 -0.100
  S1: 推定 +0.138   正解 +0.150
```

```
S2: 推定 -0.031    正解 -0.050
F2 の部品交換後の段差: 推定 +0.182    正解 +0.180
```

2.4　解説

- **いまのやり方。**ウェハーごとに、測った全装置の中央値との差を補正値とすると、米国の 2 台（U1、U2）と F2 の誤差が大きい（+0.056、+0.064、+0.072）。W3 の変化と F2 の段差を、装置のクセと取り違えているためである。本編第 5 章の長谷部の指摘に当たる。
- **一周の差。**W3 だけが z = +7.4 で有意になり、ほかの 3 枚は偶然の範囲に収まる。本編第 5 章の表の値（−0.009、+0.009、+0.210、+0.040）と一致する。
- **区間の特定。**W3 の 3 区間の変化は、ほかのウェハーが同じ区間を通らないため、和しか決まらない。そこで、変化を 1 つの区間に置いた 3 つの候補を比べ、残差平方和が最も小さい台湾から米国の区間（+0.230 nm）を選ぶ。
- **装置のクセ。**推定値は、正解から 0.02 nm 程度の範囲に収まる。F2 の部品交換後の段差も +0.182 nm と推定される。

2.5　実装上の注意

実装上の注意　W3 の変化を 1 つの区間に置くのは、「変化は 1 区間で起きた」という仮定である。2 区間以上で変化が起きた場合は、候補の組み合わせを増やすか、同じ区間を別のウェハーにも運ばせて、区間ごとの変化を直接推定できる計画にする。また、一周の差が偶然の範囲に収まったウェハーを「変わっていない」とみなす仮定は、報告書に明記し、温度と湿度の記録など数学の外の手段で確かめる（本編第 4 章）。

2.6　参照

補助文書 02 第 3〜5 章。A. Singer（2011）、R. Kenyon（2011）。

第 3 章　測り方の計画、CUSUM とカルマンフィルタ ― ch4_design_ch5_cusum.py

3.1　背景

出典：本編第 4 章（測り方の計画）、第 5 章（F2 の段差）。測る前に、案 A、案 B、案 C の計画で、未知数をすべて区別できるかを、行列の階数で確かめる。また、府中の 2 台を毎日同じ監視用のウェハーで測った差の時系列から、F2 の段差がいつ起きたかを見つける。

3.2　コード

```
"""第 4 章：測り方の計画で、装置のクセとウェハーの変化を区別できるか（行列の階数と D 基準）
```

```
第 5 章：府中の 2 台（F1, F2）を毎日同じ監視用ウェハーで測った差から、F2 の段差がいつ起きたかを見つける（CUSUM、カルマンフィル
タ）"""
import numpy as np
tools=['F1','F2','T1','T2','U1','U2','S1','S2']; site={'F':['F1','F2'],'T':['T1','T2'],'U':['U1','U2'],'S':['S1','S2']}
def design_matrix(routes):
    cols=[f'cd_{w}' for w in routes]+[f'off_{t}' for t in tools[1:]]
    legs=[]
    for w,r in routes.items():
        for k in range(1,len(r)): legs.append((w,k))
    cols+= [f'leg_{w}_{k}' for (w,k) in legs]
    X=[]
    for w,r in routes.items():
        for k,s in enumerate(r):
            for t in site[s]:
                row=np.zeros(len(cols)); row[cols.index(f'cd_{w}')]=1
                if t!='F1': row[cols.index(f'off_{t}')]=1
                for kk in range(1,k+1): row[cols.index(f'leg_{w}_{kk}')]=1
                X.append(row)
    return np.array(X),cols
designs={
 '案 A：1 枚のウェハーで一周（府中→台湾→米国→上海→府中）':{'W1':['F','T','U','S','F']},
 '案 B：4 枚（W1 府中→台湾→府中, W2 府中→米国→府中, W3 府中→台湾→米国→府中, W4 府中→上海→府中）':{'W1':
['F','T','F'],'W2':['F','U','F'],'W3':['F','T','U','F'],'W4':['F','S','F']},
 '案 C：4 枚（すべて往復のみ。W3 も府中→台湾→府中）':{'W1':['F','T','F'],'W2':['F','U','F'],'W3':['F','T','F'],'W4':
['F','S','F']},
}
print('■ 未知数（ウェハーの真値、装置のクセ、輸送区間ごとのウェハーの変化）を、すべて区別できるか')
for name,r in designs.items():
    X,c=design_matrix(r); rk=np.linalg.matrix_rank(X); n=len(c)
    ok=rk==n
    D=np.linalg.slogdet(X.T@X)[1]/n if ok else float('nan')
    print(f'  {name}\n     未知数 {n}, 階数 {rk} → {"区別できる" if ok else "区別できない（自由度 "+str(n-rk)+" が残る）"}', f'D
基準（1 未知数あたり）{D:.3f}' if ok else '')
# ---- 第 5 章：F2-F1 の差の時系列（毎日、監視用ウェハーを両方で測る） ----
rng=np.random.default_rng(11)
days=np.arange(80); true=0.12+0.18*(days>=40); sd=0.02*np.sqrt(2)
x=true+rng.normal(0,sd,len(days))
mu0=np.mean(x[:20]); k=0.045; h=0.15   # 基準期間の平均、許容幅、判定の閾値（nm）
S=0; alarm=None
for d,v in zip(days,x):
    S=max(0,S+(v-mu0-k))
    if S>h and alarm is None: alarm=d
print(f'\n■ CUSUM：基準期間の平均 {mu0:+.3f} nm、警報の日 = {alarm}日目（段差を仕込んだ日 = 40 日目）')
# 変化点の推定（警報の前で、累積和が最後に 0 だった日）
S=0; last0=0
for d,v in zip(days,x):
    S=max(0,S+(v-mu0-k))
    if S==0: last0=d
    if d==alarm: break
print(f'  変化が始まった日の推定 = {last0+1}日目')
# カルマンフィルタ（ゆっくり変わる水準を追う）
q=0.003**2; r=sd**2; m=mu0; P=0.01; est=[]
for v in x:
    P+=q; K=P/(P+r); m=m+K*(v-m); P=(1-K)*P; est.append(m)
print(f'  カルマンフィルタの推定：39 日目 {est[39]:+.3f}、45 日目 {est[45]:+.3f}、79 日目 {est[79]:+.3f}（正解：段差前 +0.120、段差
後 +0.300）')
```

3.3　実行結果

```
■ 未知数（ウェハーの真値、装置のクセ、輸送区間ごとのウェハーの変化）を、すべて区別できるか
  案 A：1 枚のウェハーで一周（府中→台湾→米国→上海→府中）
     未知数 12, 階数 9 → 区別できない（自由度 3 が残る）
  案 B：4 枚（W1 府中→台湾→府中, W2 府中→米国→府中, W3 府中→台湾→米国→府中, W4 府中→上海→府中）
     未知数 20, 階数 17 → 区別できない（自由度 3 が残る）
  案 C：4 枚（すべて往復のみ。W3 も府中→台湾→府中）
     未知数 19, 階数 16 → 区別できない（自由度 3 が残る）
```

```
■ CUSUM：基準期間の平均 +0.126 nm、警報の日 = 41 日目（段差を仕込んだ日 = 40 日目）
  変化が始まった日の推定 = 40 日目
  カルマンフィルタの推定：39 日目 +0.117、45 日目 +0.199、79 日目 +0.305（正解：段差前 +0.120、段差後 +0.300）
```

3.4　解説

- **行列の階数。**案 A は未知数 12 個に対し階数 9 で、自由度 3 が残る（本編第 4 章のリンの数え上げと一致）。案 B と案 C も、どのウェハーがどの輸送で変わってもよいとすると、自由度 3 が残る。拠点に届いたウェハーがそろって変わった場合と、その拠点の装置のクセが、同じ測定値を生むためである。本編では、この結果を受けて、仮定（一周の差が 0 に戻ったウェハーは変わっていない）を置く。
- **CUSUM。**基準期間（最初の 20 日）の平均+0.126 nm からのずれを、許容幅 0.045 nm を引きながら累積し、しきい値 0.15 nm を超えた 41 日目に警報を出す。累積が最後に 0 だった日から、変化の始まりを 40 日目と推定する。
- **カルマンフィルタ。**2 台の差の水準を追い、39 日目+0.117、45 日目+0.199、79 日目+0.305 と推定する。段差の直後は、推定が新しい水準にゆっくり近づく。

3.5　実装上の注意

実装上の注意　CUSUM の許容幅としきい値は、誤警報の頻度と、変化を見つけるまでの日数の釣り合いで決める。本コードの値は、物語の架空のデータに合わせた例である。カルマンフィルタの状態のばらつき（q）を大きくすると段差への追従は速くなるが、毎日の推定のぶれも大きくなる。段差のような急な変化には、CUSUM で変化点を見つけてから、カルマンフィルタを変化点の後で始め直す組み合わせが実用的である。

3.6　参照

補助文書 02 第 6・7 章。E. S. Page（1954）、R. E. Kalman（1960）。

第 4 章　規制の両立と実物オプション ― ch6_ch7.py

4.1　背景

出典：本編第 6 章・第 7 章。中国の先端の工場のお客さま向けの保守の時間の条件が両立するかを、線形計画法とファルカスの補題で判定する。数値でない条件（中国の当局の通知、社内規程、法務の判断、米国の規制）が両立するかを、z3 で判定し、矛盾の原因の最小の組を取り出す。最後に、分けるか待つかを、実物オプションで比べる。

4.2　コード

```
"""第 6 章：両立の判定（架空の数値と、物語の中の法務の判断）　第 7 章：分けるか待つか（実物オプション）"""
import numpy as np, z3
from scipy.optimize import linprog
# ---- 数値の条件：中国の先端の工場の顧客向けの保守（月あたりの技術者の時間） ----
# 変数 x=[上海の非米国人の技術者の時間, 府中から派遣する技術者の時間, 米国人の技術者の時間]
rows=['顧客との契約：必要な保守の時間　x1+x2+x3 ≥ 1800','上海の非米国人の技術者の上限　x1 ≤ 1200','府中からの派遣の上限　x2 ≤
400','米国の規制（許可なし）：米国人の時間　x3 ≤ 0']
A=np.array([[-1,-1,-1],[1,0,0],[0,1,0],[0,0,1]],float); b=np.array([-1800,1200,400,0],float)
def check(A,b,rows,title):
    r=linprog(np.zeros(3),A_ub=A,b_ub=b,bounds=[(0,None)]*3,method='highs')
    print('■',title)
    if r.status==0: print('  両立する。例：',np.round(r.x)); return
    m=len(b); Aeq=np.vstack([A.T,b[None,:]])
    # x>=0 なので、証明書は y>=0, A^T y >= 0, b^T y < 0
    c=linprog(np.ones(m),A_ub=-A.T,b_ub=np.zeros(3),A_eq=b[None,:],b_eq=[-1],bounds=[(0,None)]*m,method='highs')
    y=c.x/np.max(c.x)
    print('  両立しない。証明書（各条件に掛ける重み）：')
    for yi,row in zip(y,rows):
        if yi>1e-9: print(f'    {yi:.2f} × [{row}]')
    print(f'  重みをつけて足すと：0 ≤ {y@b:.0f}（矛盾）')
check(A,b,rows,'先端の工場の顧客（月 1,800 時間の保守が必要）')
b2=b.copy(); b2[0]=-900
check(A,b2,['顧客との契約：必要な保守の時間 ≥ 900']+rows[1:],'先端でない工場の顧客（月 900 時間）')
# ---- 数値でない条件 ----
v={k:z3.Bool(k) for k in ['raw_export','by_central','central_has_us','us_support_adv','by_china_team','diff_only']}
R={
 'C1 中国：上海の生の計測データは、安全評価が済むまで国外へ出さない':z3.Not(v['raw_export']),
 'K1 社内規程：中国の先端の工場向けの補正値も、府中の全社チームが計算する':v['by_central'],
 'K2 府中の全社チームには、米国子会社の米国人の技術者が含まれる':v['central_has_us'],
 'L1 法務の判断（物語の設定）：先端の工場の装置の補正値を米国人が計算すると、保守の支援に当た
る':z3.Implies(z3.And(v['by_central'],v['central_has_us']),v['us_support_adv']),
 'U1 米国の規制：許可がない限り、米国人は先端の工場の保守を支援しない':z3.Not(v['us_support_adv']),
 'M1 補正値の計算には、生のデータか、差の集計のどちらかが必要':z3.Or(v['raw_export'],v['diff_only']),
 'M2 補正値は、府中の全社チームか、中国向けの専任チームのどちらかが計算する':z3.Or(v['by_central'],v['by_china_team']),
}
def sat(rs,title):
    s=z3.Solver(); tags={}
    for k,e in rs.items():
        t=z3.Bool('t:'+k); tags[str(t)]=k; s.assert_and_track(e,t)
    print('\n■',title)
    if s.check()==z3.sat:
        m=s.model(); print('  両立する：',{str(d):m[d] for d in m.decls() if not str(d).startswith('t:')}); return
    core=[tags[str(c)] for c in s.unsat_core()]
    for k in list(core):
        t=[c for c in core if c!=k]; s2=z3.Solver(); [s2.add(rs[c]) for c in t]
        if s2.check()==z3.unsat: core=t
    print('  両立しない。原因になっている最小の組：'); [print('   -',c) for c in core]
sat(R,'いまの体制')
R2={k:e for k,e in R.items() if not k.startswith('K1')}
R2["K1' 社内規程の見直し：中国の先端の工場向けの補正値は、米国人を含まない専任チームが、差の集計だけから計算す
る"]=z3.And(z3.Not(v['by_central']),v['by_china_team'],v['diff_only'])
sat(R2,'専任チームに分けた場合')
# ---- 第 7 章 実物オプション（単位：億円、割引率 5%） ----
r=0.05; d=1/(1+r); cC,cR,K,s=12.0,2.0,40.0,3.0
pv=lambda a:a*(1+r)/r; VR=pv(cR); Vs=K+pv(s)
def solve(q):
    V=Vs
    for _ in range(3000):
        w=cC+d*(q*VR+(1-q)*V); V=min(Vs,w)
    return V,w
lo,hi=0,1
for _ in range(60):
    mid=(lo+hi)/2; _,w=solve(mid)
    lo,hi=(mid,hi) if w>Vs else (lo,mid)
print(f'\n■ 実物オプション：いま分ける {Vs:.1f}、解消後も統合のまま {VR:.1f}、境目の確率 q* ≈ {hi:.3f}')
for q in [0.10,0.20,0.30]:
```

```
    V,w=solve(q); print(f'  q={q:.2f}：待つ場合の費用 {w:.1f}、分ける場合 {Vs:.1f}、待つことの価値 {max(0,Vs-w):.1f}')
```

4.3　実行結果

```
■ 先端の工場の顧客（月 1,800 時間の保守が必要）
  両立しない。証明書（各条件に掛ける重み）：
    1.00 × [顧客との契約：必要な保守の時間　x1+x2+x3 ≥ 1800]
    1.00 × [上海の非米国人の技術者の上限　x1 ≤ 1200]
    1.00 × [府中からの派遣の上限　x2 ≤ 400]
    1.00 × [米国の規制（許可なし）：米国人の時間　x3 ≤ 0]
  重みをつけて足すと：0 ≤ -200（矛盾）
■ 先端でない工場の顧客（月 900 時間）
  両立する。例： [1200.    0.    0.]

■ いまの体制
  両立しない。原因になっている最小の組：
   - K1 社内規程：中国の先端の工場向けの補正値も、府中の全社チームが計算する
   - K2 府中の全社チームには、米国子会社の米国人の技術者が含まれる
   - L1 法務の判断（物語の設定）：先端の工場の装置の補正値を米国人が計算すると、保守の支援に当たる
   - U1 米国の規制：許可がない限り、米国人は先端の工場の保守を支援しない

■ 専任チームに分けた場合
  両立する： {'by_central': False, 'by_china_team': True, 'diff_only': True, 'central_has_us': True, 'us_support_adv':
False, 'raw_export': False}

■ 実物オプション：いま分ける 103.0、解消後も統合のまま 42.0、境目の確率 q* ≈ 0.122
  q=0.10：待つ場合の費用 104.3、分ける場合 103.0、待つことの価値 0.0
  q=0.20：待つ場合の費用 84.0、分ける場合 103.0、待つことの価値 19.0
  q=0.30：待つ場合の費用 72.0、分ける場合 103.0、待つことの価値 31.0
```

4.4　解説

- **ファルカスの補題。**両立しない場合、条件に掛ける重み（証明書）を線形計画法で求め、重みをつけて足すと $0 \le -200$ になることを示す。月 200 時間の不足は、誰が計算しても同じ結論になる、という意味で「証明」である。先端でない工場（月 900 時間）では、上海の技術者だけで足りる。
- **最小の矛盾の組。**z3 の矛盾の核から、条件を 1 つずつ外しても満たせないかを確かめ、外せない条件だけを残す。K1、K2、L1、U1 の 4 つが残り、中国の当局の通知（C1）は含まれない。K1 を専任チームの形に改めると両立する。
- **実物オプション。**毎年、確率 q で規制が見直されるとし、「いま分ける」と「1 年待つ」の安いほうを選ぶ問題を、価値の反復で解く。境目の q を二分法で求めると約 0.122 になる。q = 0.20 では待つ費用 84.0 億円、待つことの価値 19.0 億円で、本編第 7 章の数値と一致する。

4.5　実装上の注意

実装上の注意　法務の判断（L1）は、物語の設定である。実際に規制を論理式に落とすときは、法務が条文と運用を確かめたうえで、どの事実が規制の対象に当たるかを式にする。z3 の最小化の方法

は、条件の数が少ない場合の簡単な方法であり、条件が多い場合は、MUS を求める専用のアルゴリズムを使う。実物オプションの費用は見積もりであり、見積もりの幅を変えて境目がどれだけ動くかを、取締役会の資料に添える。

4.6　参照

補助文書 02 第 9〜11 章。J. Farkas（1902）、M. H. Liffiton, K. A. Sakallah（2008）、A. K. Dixit, R. S. Pindyck（1994）。手引書 Vol.2 第 27 章。

第 5 章　本編の図を描くコード — figures/

数値や文字を正確に描く必要がある図は、Python（Matplotlib）で描いた。日本語の文字を描くため、Noto Sans CJK のフォントを使う。フォントの場所は、環境に合わせて書き換える。図は、スクリプトを実行した場所に、jpg で保存される。

スクリプト	描く図	本編の場所
telescoping.py	koryo2_03b_telescoping.jpg：一周の差を足し合わせると、台湾と米国の値が打ち消し合う	第 2 章（リームの説明）
two_loops.py	koryo2_04c_two_loops.jpg：基準ウェハーが 1 枚の場合と、もう 1 枚加えた場合	第 3 章（倉田教授の説明）
wafer_routes.py	koryo2_05b_wafer_routes.jpg：4 枚の基準ウェハーの経路（案 B）	第 4 章
cannot_tell.py	koryo2_05c_cannot_tell.jpg：ウェハーがそろって変わった場合と、台湾の装置のクセの場合は、測った値が同じになる	第 4 章（リンの説明）
loop_graph.py	koryo2_04b_loop_graph.jpg：一周の輪と線の上の差（本編には載せなかった補足の図）	—

実装上の注意　同じ区間を通る複数の矢印は、線の向きに垂直な方向に平行にずらして、重ならないように描いている（wafer_routes.py の lane 関数）。画像生成 AI で同じ図を作ると、文字や矢印が崩れることがあったため、Python で描いた。

付録 A　推奨ライブラリ

ライブラリ	用途
NumPy	行列の計算、最小二乗法、行列の階数、乱数による架空のデータの生成
SciPy	線形計画法（scipy.optimize.linprog、HiGHS）によるファルカスの補題の証明書の計算
z3-solver	数値でない条件の充足可能性の判定と、矛盾の核の取り出し
Matplotlib	本編の図の描画

```
pip install numpy scipy matplotlib z3-solver
python code/ch4_ch5_metrology.py
python code/ch4_design_ch5_cusum.py
python code/ch6_ch7.py
```